Predicting Stack Overflow Tags from Question Titles

Metis Data Science Bootcamp, Project 3, February 2021

Garreth Cline

A question from the data, with its tags

How do I calculate someone's age in C#?

c# .net datetime

The 10 most common tags

c# java javascript python .net

c++ jquery php android html

Slides remade in October 2026 with AI help. Every result comes from the original 2021 notebook.

Code and notebook: github.com/garrethcline/Predicting-Tags-from-StackOverflow-Using-Title-Data-

The goal

Recommend tags for a Stack Overflow question from its title alone

Suggest tags as someone writes

Recommend tags while a question is still being written.

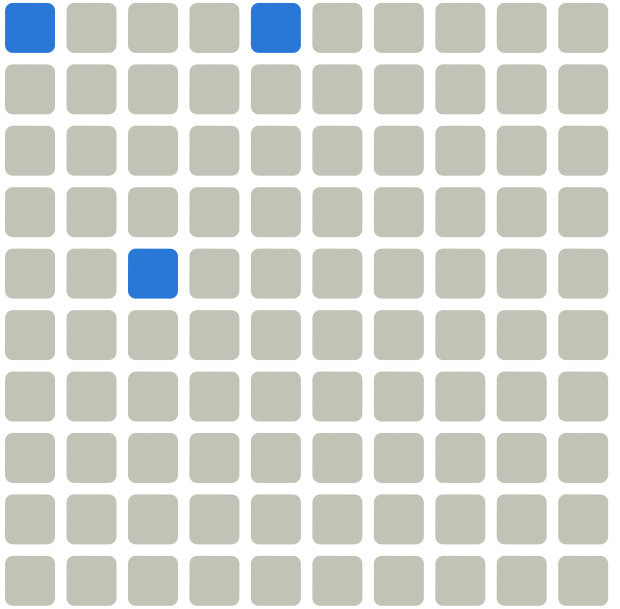
Tag old posts

Add tags to posts or other text that has none.

Several tags per question

A question can have up to five tags, so the model makes a separate yes or no call for each of the 100 most common tags.

How do I calculate someone's age in C#?



One square per tag, most common first. This question has 3 of the 100 tags.

Where the data came from

Stack Overflow's public data dump, 2010 version, loaded into SQL Server from brentozar.com



Data dump

Stack Overflow posts up to 2010, as a SQL Server database

SQL Server

Removed commas from titles and tags, stripped the HTML angle brackets from tags and deleted posts with no title

CSV export

1,000,000 posts with title, tags, score and view count, exported with SQL Server Management Studio

Python

Filtered, cleaned and modeled in a Jupyter notebook with pandas, NLTK and scikit-learn

Tools SQL Server | Python | pandas | NLTK | scikit-learn | Jupyter | 32-thread workstation

Filtering to popular questions

Well-rated, widely viewed questions with at least one common tag

1,000,000
posts exported
100% of the export

→

28,183
with a score above 50
2.8% of the export

→

27,666
with more than 10,000 views
2.8% of the export

→

25,352
with at least one top-100 tag
2.5% of the export

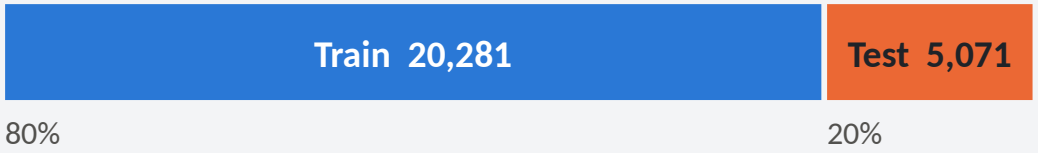
Tags

The 27,666 questions used 6,884 different tags, 82,516 times in all. I kept the 100 most common.

6,884 tags → 100 kept

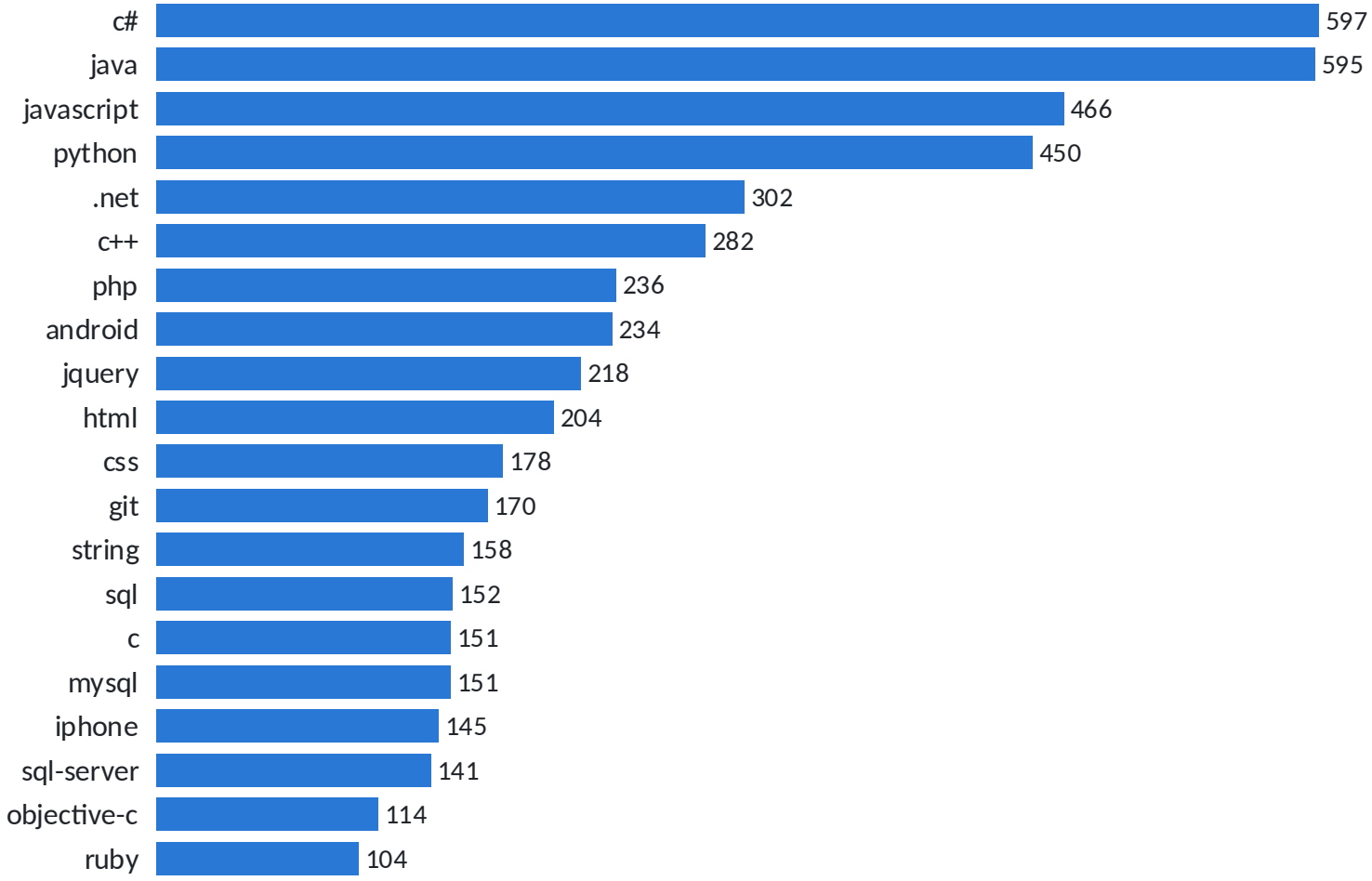
Train and test

Held out 20% of the 25,352 questions for testing.



Some tags are far more common than others

Questions per tag in the test set, for the 20 most common of the 100 tags



1.6

tags per question

of the 100, on average

12%

of questions have c#

the most common tag

14

have parsing

the least common of the 100

Cleaning the titles

One title through each step, using the notebook's own output

1

Original title

```
“How do I calculate someone’s age in C#?”
```

→

2

Punctuation removed

```
“How do I calculate someone s age in C”
```

→

3

Base forms, common words removed

```
“How I calculate someone age C”
```

→

4

TF-IDF scores

A score for each of the 1,000 most common words. One-letter words like “C” don’t count.

→

5

100 yes or no calls

One classifier per tag decides whether the title has that tag.

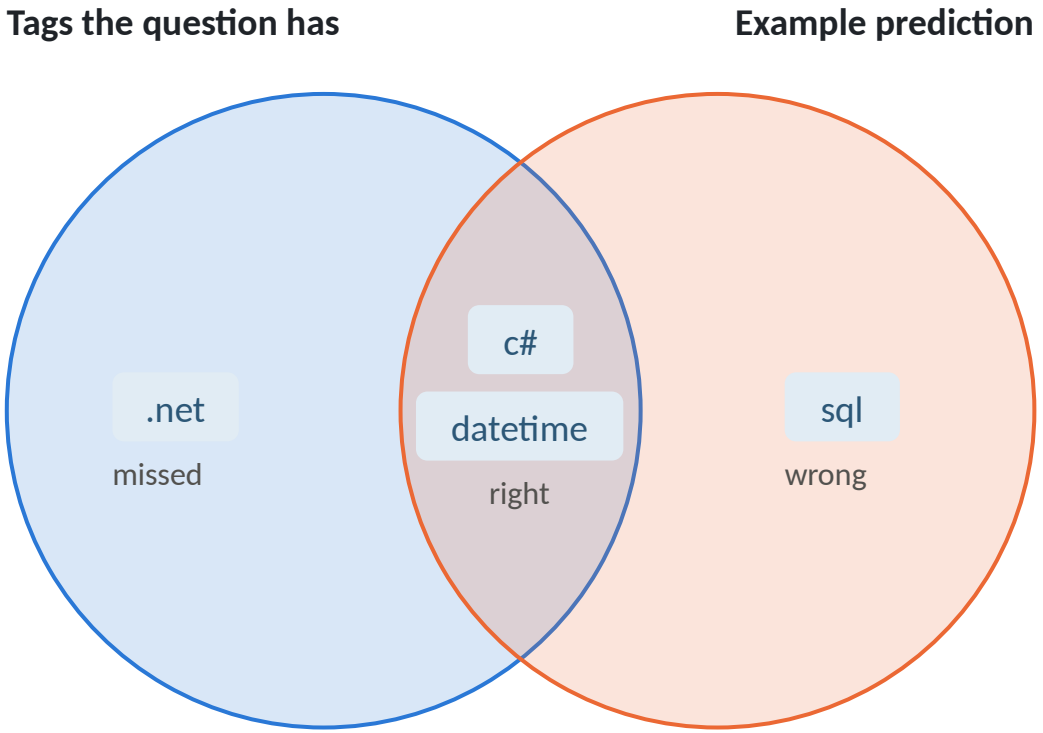
The answers it learns from c# .net datetime

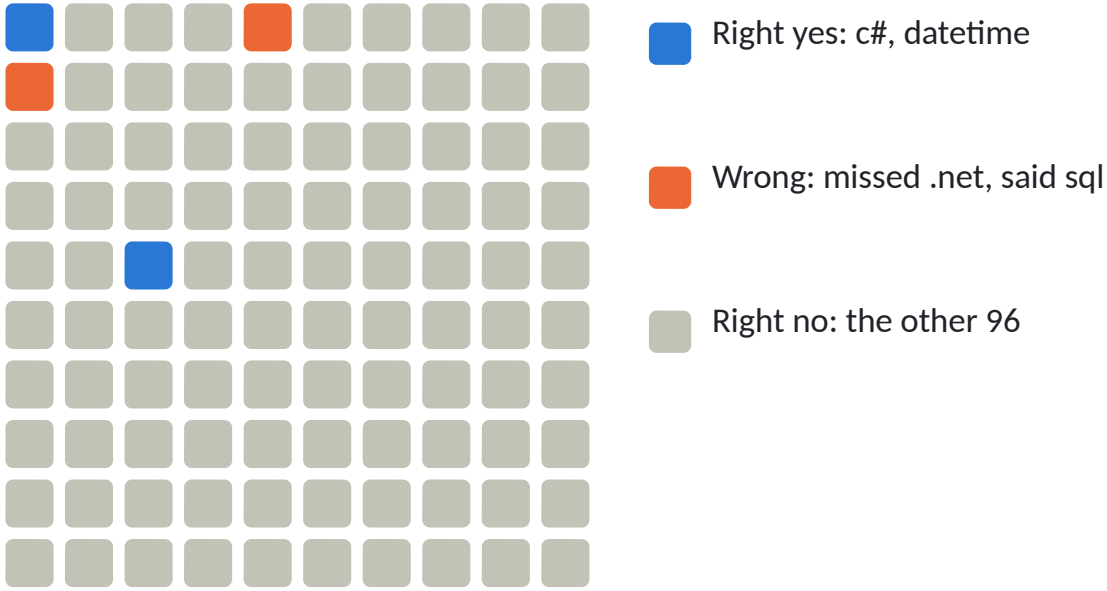
The question’s tags, split on spaces. Only the 100 most common tags were kept.

Something I noticed in 2026: capitalized common words like “How” got through, because the common-word filter only matched lower case.

How the scores work

One real question with a made-up prediction, to show the two scores





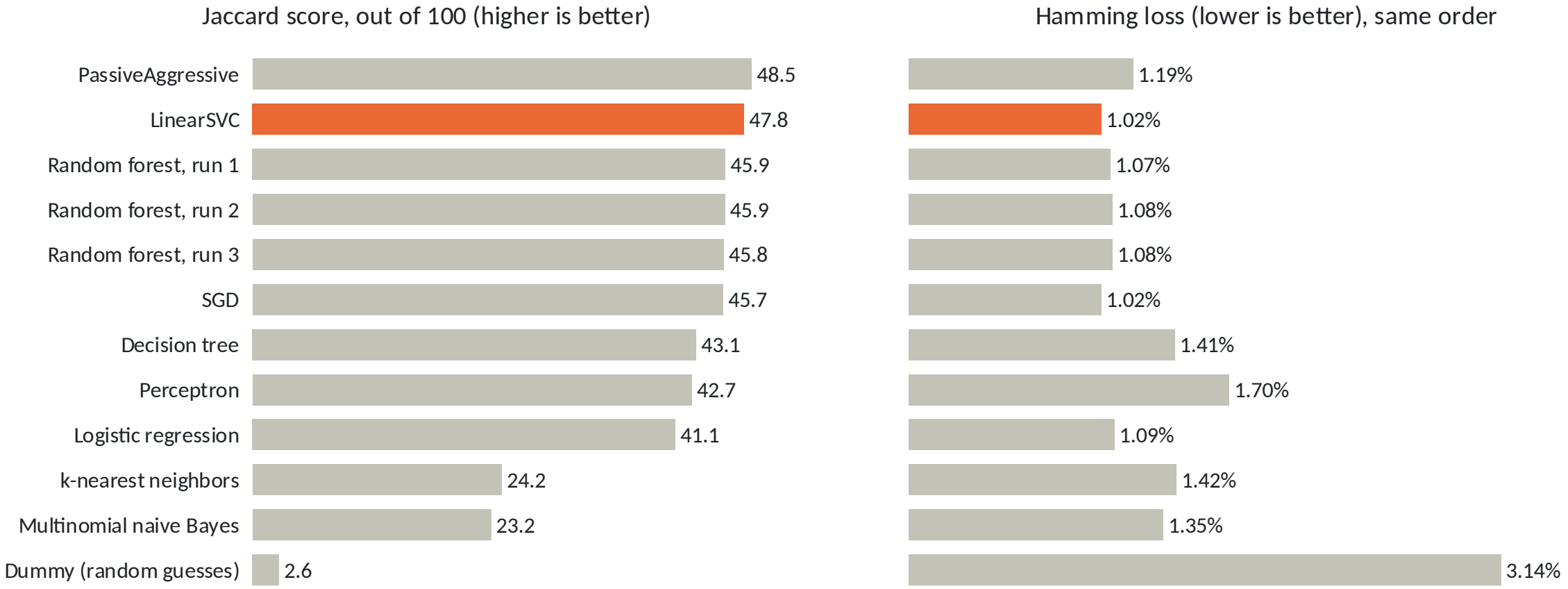
Jaccard score: 2 tags right ÷ 4 tags in total = 50 out of 100

Hamming loss: 2 wrong calls out of 100 = 2%

Most of the 100 calls are easy no's, so Hamming loss is low for every model. The Jaccard score shows the differences better.

Twelve models on the same test set

Each ran as 100 one-vs-rest yes or no classifiers, without tuning



The 2021 code labeled the three random forests 10, 100 and 1,000 trees, but all three were set to 1,000. The dummy guessed each tag at random, at that tag's rate in the training data.

Final model: LinearSVC

Lowest Hamming loss and second-highest Jaccard score of the twelve

Grid search over C

C = 1 | C = 10 | C = 100 | C = 1,000

× 100 folds = 400 fits, about four minutes on 32 threads

The best value was C = 1, the default, so the tuned model scored the same as the untuned one.

100 folds was probably more than needed; the notebook notes that 15 gave similar results with much less load.

PassiveAggressive	Jaccard 48.5 · Hamming loss 1.19%
LinearSVC	Jaccard 47.8 · Hamming loss 1.02%

47.8

Jaccard score

Dummy model: 2.6

1.02%

Hamming loss

Dummy model: 3.14%

85.5%

Precision

Share of predicted tags that were right

46.0%

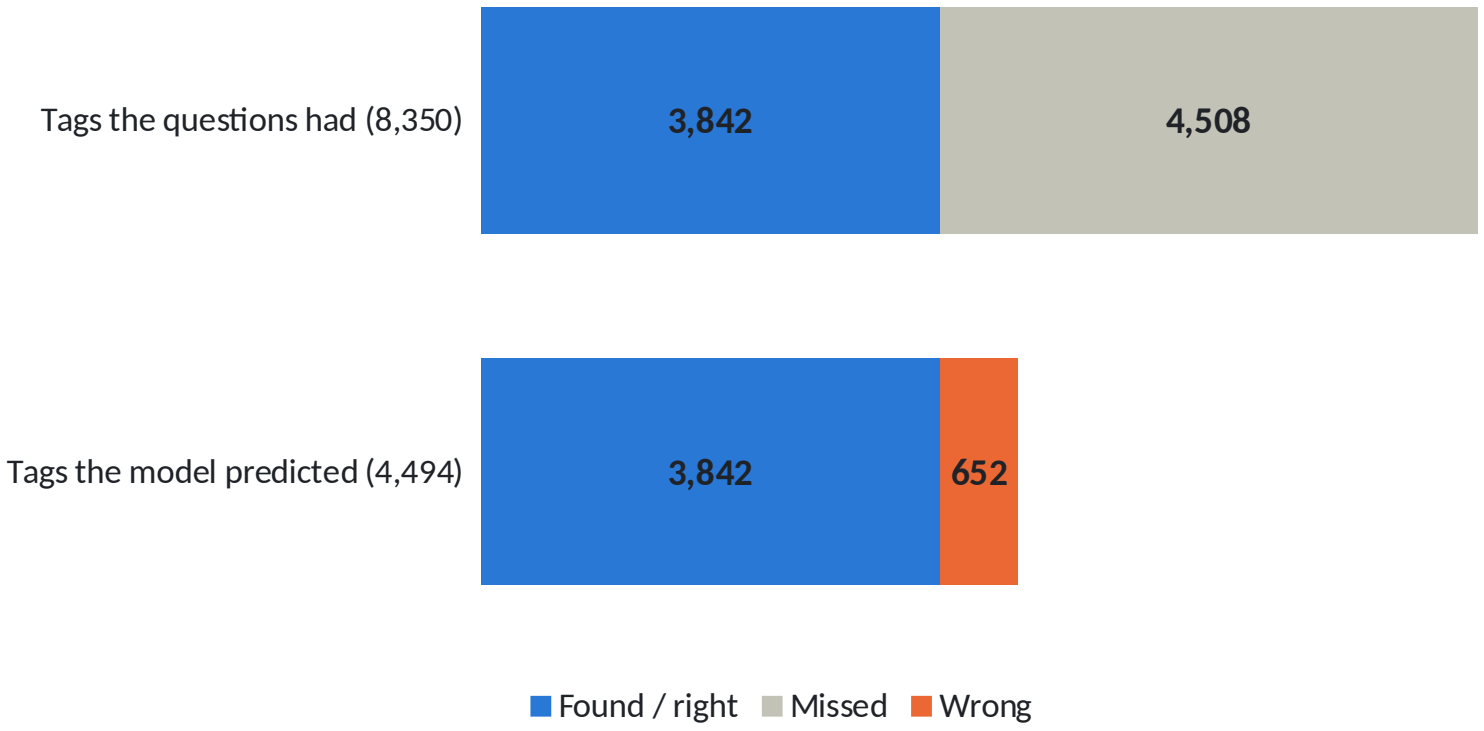
Recall

Share of actual tags the model found

Precision and recall were added in 2026, from the notebook's 100 per-tag confusion matrices.

Usually right, but it missed over half the tags

All 100 tags across the 5,071 test questions



85.5%

Precision

3,842 of the 4,494 predicted tags were right

46.0%

Recall

3,842 of the 8,350 actual tags were found

Every tag used the same default cutoff. A separate cutoff for each tag could trade some precision for more recall.

Two tags up close: git and c#

Each tag has its own yes or no model. Its confusion matrix shows where it went wrong.

	Model said yes	Model said no
Has git (170)	148 found	22 missed
Doesn't (4,901)	4 wrong	4,897 correct no

Precision 97% · **Recall 87%**

"git" was the strongest word for this tag, and the model missed only 22 of 170.

	Model said yes	Model said no
Has c# (597)	183 found	414 missed
Doesn't (4,474)	91 wrong	4,383 correct no

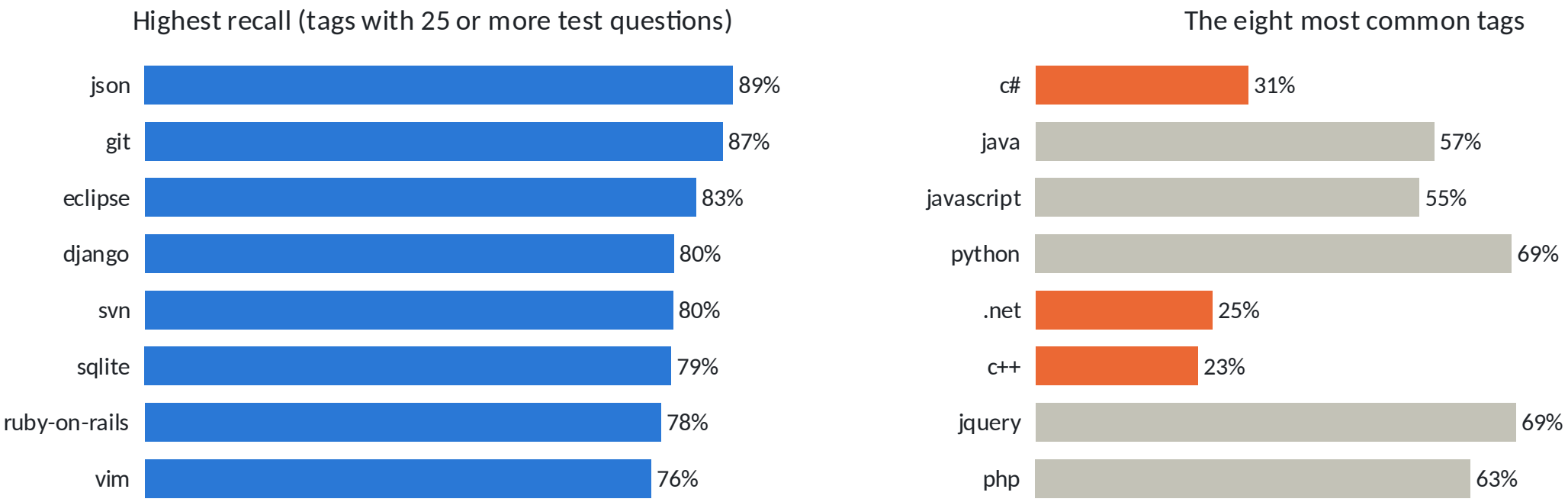
Precision 67% · **Recall 31%**

The cleanup dropped "C#" from titles when it was capitalized (slide 14).

Precision = found ÷ (found + wrong) **Recall** = found ÷ (found + missed)

Where it worked and where it didn’t

Recall: the share of questions with the tag that the model caught



Never predicted correctly (11 tags), mostly broad topics

oop | class | file | math | coding-style | language-agnostic | file-io | types | scripting | cocoa-touch | visual-studio-2008

What the model learned

The eight title words with the most weight for each tag, strongest first

git	git	github	commit	branch	repo	repository	remote	master
android	android	activity	webview	edittext	textview	intent	listview	emulator
python	python	matplotlib	numpy	dict	django	module	none	modules
java	java	jpa	spring	junit	jvm	hibernate	servlet	arraylist
css	css	border	divs	div	font	span	align	overflow
sql	sql	procedure	oracle	inner	query	t-sql	on	by
iphone	iphone	uiscrollview	uitableview	uiview	uibutton	uiimage	uilabel	ios
c++	std	cc	boost	c++	stl	virtual	qt	template

For most tags the strongest word is the tag's own name. For c++ it is "std"; the next slide shows why.

What the cleanup did to C# and C++

Found in 2026 while reviewing the 2021 code

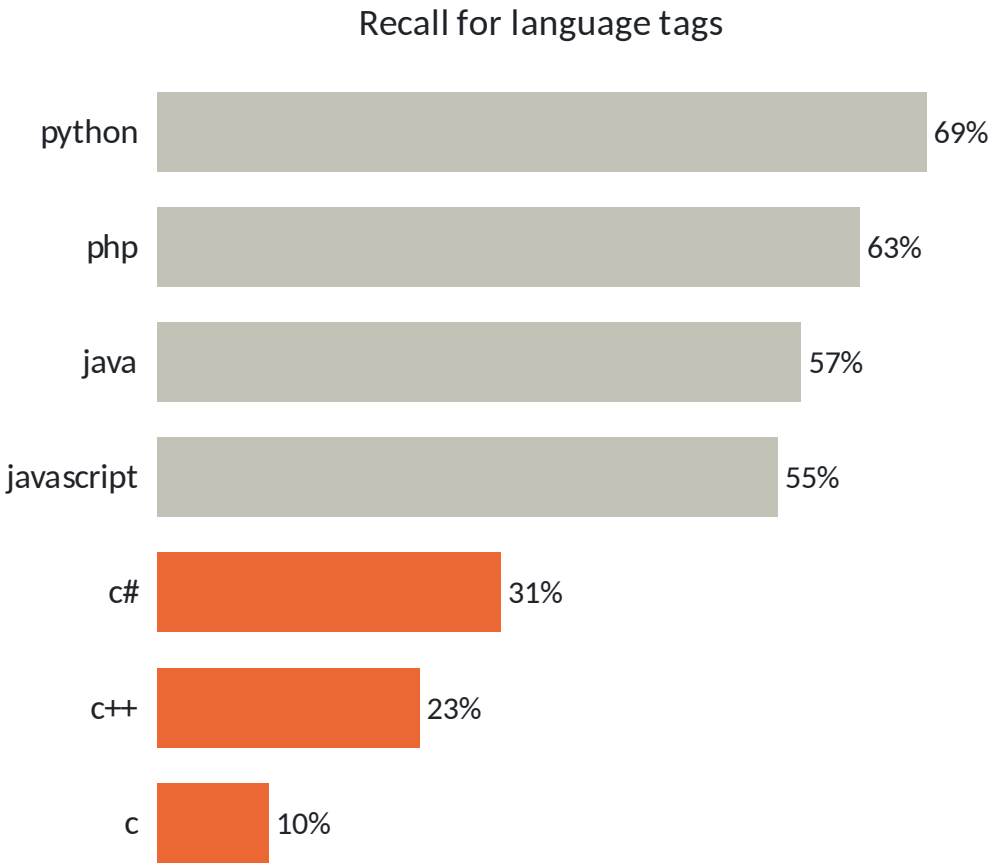
In the title	After punctuation removal	After TF-IDF
C#	C	*nothing left*
C++	C	*nothing left*
C/C++	cc	kept as “cc”

Notebook output for one title

How do I calculate someone’s age in C#?

How do I calculate someone s age in C

This is probably part of why these three tags scored lowest. The fix: match tag names in any case before removing punctuation, and let TF-IDF keep one-letter words.



What I would change

Five things I would do differently, from the results above

Use the question body

Titles often don’t name the tool. I left the body out in 2021 because it needed much more memory and I wanted more rows.

Fix the cleanup

Keep C#, C++ and one-letter words, and make the common-word filter ignore case.

One cutoff per tag

Trade some precision (85.5%) for more recall (46.0%).

Newer data

The database ends in 2010, and the filters kept only popular questions.

More tags

Only the 100 most common tags were modeled; 6,784 were left out.

Code, notebook and these slides: github.com/garrethcline/Predicting-Tags-from-StackOverflow-Using-Title-Data-